

API List

API	Request Body	Response	Description
GET /accounts	(none) "comp_code": "string", //superadmin	200 OK 401 Invalid	Lists all accounts belonging to the same company
GET /accounts/{id}	(none)	200 Account Found 401 Invalid 403 Forbidden 404 Not Found	Get account by ID
POST /accounts	{ "comp_code": "string", //superadmin "emp_id": "string", "fullname": "string", "username": "string", "email": "user@example.com", "password": "stringst", "department_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "role_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "user_type_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "status": "active" }	201 Account created successfully 400 Bad Request 401 Invalid 403 Forbidden 409 Conflict 422 Validation Failed	Create new account
PUT /accounts/{id}	{ "comp_code": "string", "emp_id": "string", "fullname": "string", "username": "string", "email": "user@example.com", "password": "stringst", "department_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "role_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "user_type_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "status": "active" } (only include elements needed to modify)	200 Account updated 400 Bad Request 401 Invalid 403 Forbidden 404 Not Found 409 Conflict	Update account by ID
PATCH /accounts/{id}	(none)	200 Account updated 400 Bad request 401 Invalid	Disable account by ID

		403 Forbidden 404 Not Found 409 Conflict	
DELETE /accounts/{id}	(none)	204 Account deleted 400 Bad request 401 Invalid 403 Forbidden 404 Not Found 409 Conflict	Permanently delete account by ID
GET /departments	(none) "comp_code": "string", //superadmin	200 OK 401 Invalid	Lists all departments belonging to the same company
GET /departments/{id}	(none)	200 Department Found 401 Invalid 403 Forbidden 404 Not Found	Get department by ID
POST /departments	{ "comp_code": "string", //superadmin "department_name": "string", "description": "string", "status": "active" }	201 Department created successfully 400 Bad Request 401 Invalid 403 Forbidden 409 Conflict 422 Validation Failed	Create new department
PUT /departments/{id}	{ "comp_code": "string", "department_name": "string", "description": "string", "status": "active" } (only include elements needed to modify)	200 Department updated 400 Bad Request 401 Invalid 403 Forbidden 404 Not Found 409 Conflict	Update department by ID
PATCH /departments/{id}	{ "comp_code": "string", "status": "active" }	200 Department updated 400 Bad request 401 Invalid 403 Forbidden 404 Not Found 409 Conflict	Disable department by ID
DELETE /departments/{id}	(none)	204 Department deleted 400 Bad request 401 Invalid 403 Forbidden 404 Not Found 409 Conflict	Permanently delete department by ID

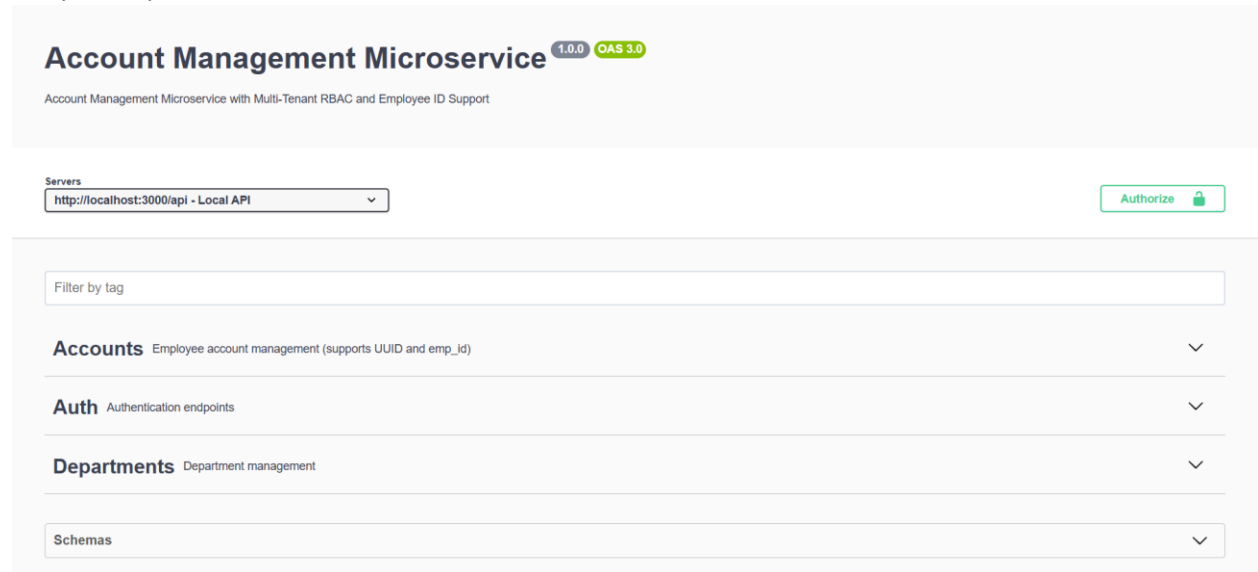
Configuration

Title	Config Name	Default Value	Description
Service Port	PORT	3000	Port the microservice listens on
Database Host	DB_HOST	localhost	PostgreSQL database host
Database Port	DB_PORT	5432	PostgreSQL database port
Database Name	DB_NAME	account_db	Name of the PostgreSQL database
Database User	DB_USER	postgres	Database username
Database Password	DB_PASS	postgres	Database password
JWT Secret	JWT_SECRET	<i>(none)</i>	Secret key used to sign and verify JWT tokens
JWT Expiration	JWT_EXPIRES_IN	1d	JWT token expiration duration
Bcrypt Salt Rounds	BCRYPT_SALT_ROUNDS	10	Cost factor used for password hashing
Auth Microservice URL	MICROSERVICE_URL	http://localhost:3000/api/login	URL of the account/auth microservice used to validate credentials
JWT Secret (Mobile)	JWT_SECRET	<i>(none)</i>	Secret key used to sign mobile login JWT tokens
Mobile Token Expiration	TOKEN_EXPIRY	30m	Expiration time for JWT issued by mobile login service

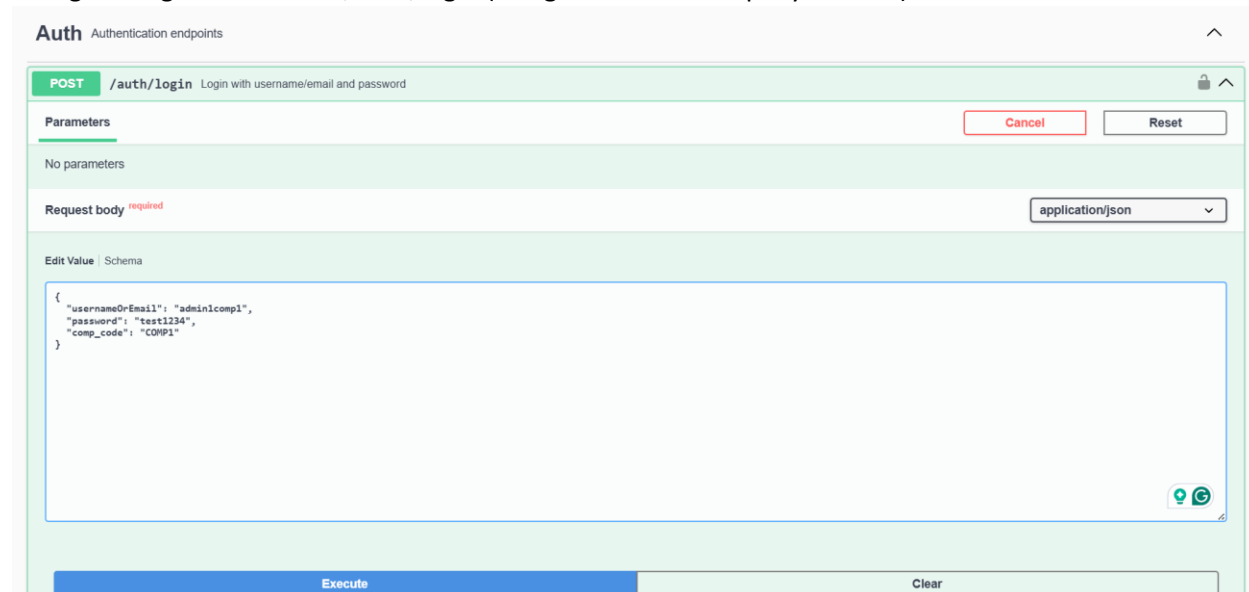
Account Management Microservice on Swagger UI

LOGIN/LOGOUT

1. Open <http://localhost:3000/docs/#/>.



2. Login using Auth -> POST /auth/login (using Admin1 of company COMP1).



3. If login is successful, token will be displayed in the response body.

[illegible]

4. Copy token and paste into Authorize button on top right of page (can also logout from here).

 **Swagger**
Powered by SMARTBEAR

Account Management Microservice

Account Management Microservice with Multi-Tenant RBAC and Employee ID Support

Servers

http://localhost:3000/api - Local API

Authorize 

Available authorizations

bearerAuth (http, Bearer)

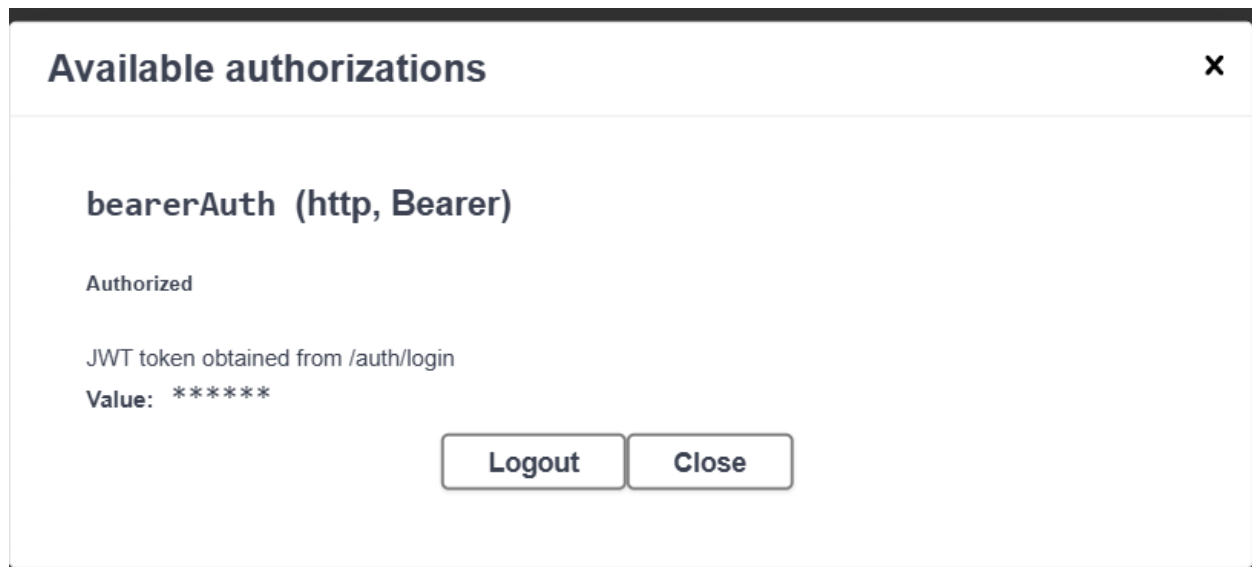
JWT token obtained from /auth/login

Value:

1eQ_UHiqOwFgga5JBYQA-rd

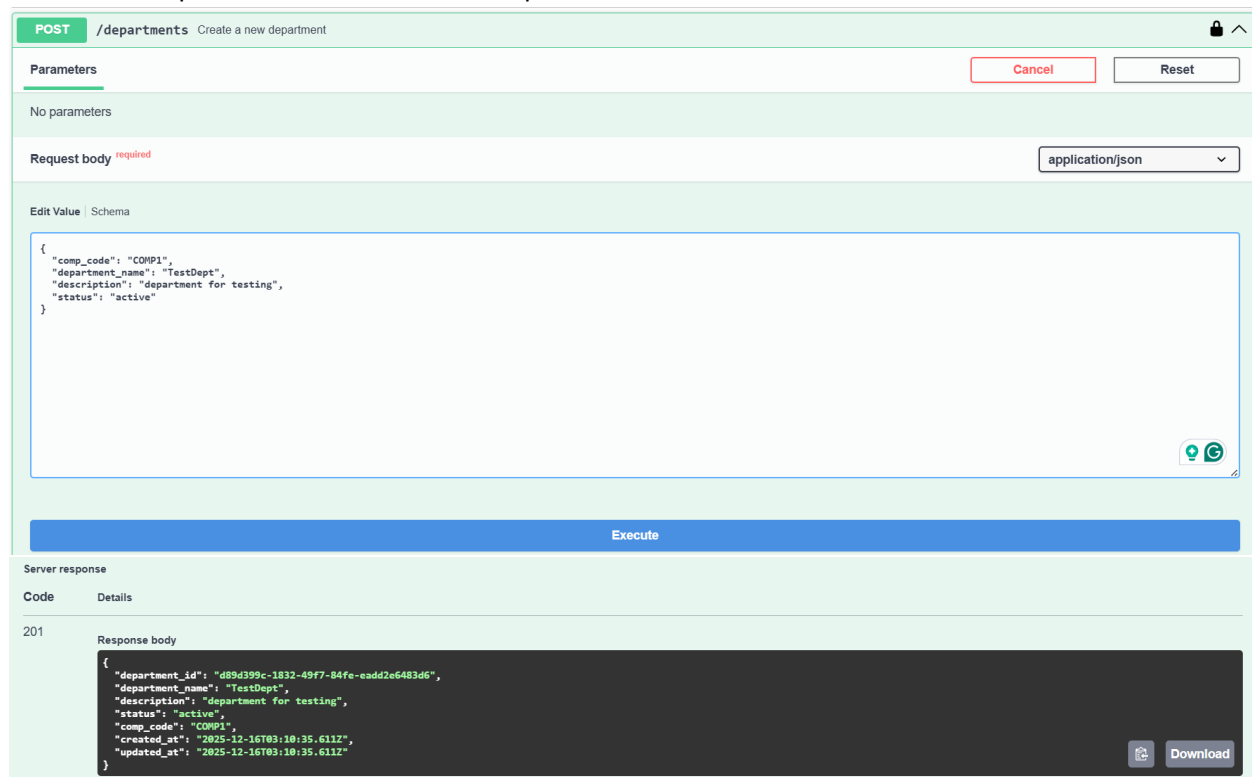
Authorize

Close



DEPARTMENTS

1. To create a new department, go to Departments -> POST /departments. A department within COMP1 named TestDept will be created as an example.



4. To update any details about the department, copy the department id and paste it into PUT /departments/{id}. In the request body, include any department details that need changing.
Note: comp_code is required to make edits

The screenshot shows a REST client interface for a PUT request to the endpoint `/departments/{id}` with the subtext "Update a department by ID". The interface includes a "Parameters" section with a table for the request parameters. The "id" parameter is required, of type "string", and is located at the path `d89d399c-1832-49f7-84fe-eadd2e6483d6`. The "Request body" section is set to "application/json" and contains a JSON object: `{ "comp_code": "COMP1", "description": "department for testing edited" }`. The "Server response" section shows a 200 status code and a detailed response body: `{ "department_id": "d89d399c-1832-49f7-84fe-eadd2e6483d6", "department_name": "TestDept", "description": "department for testing edited", "status": "active", "comp_code": "COMP1", "created_at": "2025-12-16T03:10:35.611Z", "updated_at": "2025-12-16T03:23:48.028Z" }`. Buttons for "Cancel", "Reset", and "Download" are visible.

Name	Description
id * required	
string	
(path)	d89d399c-1832-49f7-84fe-eadd2e6483d6

Request body required application/json

```
{
  "comp_code": "COMP1",
  "description": "department for testing edited"
}
```

Server response

Code	Details
200	<pre>{ "department_id": "d89d399c-1832-49f7-84fe-eadd2e6483d6", "department_name": "TestDept", "description": "department for testing edited", "status": "active", "comp_code": "COMP1", "created_at": "2025-12-16T03:10:35.611Z", "updated_at": "2025-12-16T03:23:48.028Z" }</pre>

5. To update only the status of a department, copy the department id and paste it into PATCH /departments/{id}/status. In the request body, change the status to either active/inactive.
Note: comp_code is required to make edits, only SUPERADMIN can update department statuses

The screenshot shows a REST client interface for a PATCH request to the endpoint `/departments/{id}/status` with the subtext "Update department status (active/inactive)". The interface includes a "Parameters" section with a table for the request parameters. The "id" parameter is required, of type "string", and is located at the path `d89d399c-1832-49f7-84fe-eadd2e6483d6`. The "Request body" section is set to "application/json" and contains a JSON object: `{ "comp_code": "COMP1", "status": "inactive" }`. The "Server response" section is currently empty. Buttons for "Cancel", "Reset", and "Download" are visible.

Name	Description
id * required	
string	
(path)	d89d399c-1832-49f7-84fe-eadd2e6483d6

Request body required application/json

```
{
  "comp_code": "COMP1",
  "status": "inactive"
}
```


6. To delete a department, copy the department id and paste it into DELETE /departments/{id}.

Note: only SUPERADMIN can delete departments

DELETE

/departments/{id}

Delete a department by ID



Parameters

Cancel

Name	Description
id ★ required	Department ID
string (path)	d89d399c-1832-49f7-84fe-eadd2e6483d6

Execute

ACCOUNTS

1. To create a new account, go to Accounts -> POST /accounts. An employee account from the department TestDept in COMP1 named TestEmp will be created as an example.

Note: management must obtain user_type_id and role_id via SQL

POST

/accounts

Create a new employee account



Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{
  "comp_code": "COMP1",
  "emp_id": "EMP000",
  "fullname": "TestEmp",
  "username": "testemp",
  "email": "testemp@comp1.com",
  "password": "test1234",
  "department_id": "d89d399c-1832-49f7-84fe-eadd2e6483d6",
  "role_id": "913f5fb0-58ed-4a88-bcf0-leadc7e756d3",
  "user_type_id": "fcb3de16-32a1-4fc8-8b45-f9e6bb8964ca"
}
```



Execute

Server response

Code

Details

201

Response body

```
{
  "id": "99f5609f-ef5d-441b-a8da-a6a65500b1c3",
  "emp_id": "EMP000",
  "fullname": "TestEmp",
  "username": "testemp",
  "email": "testemp@comp1.com",
  "department_id": "d89d399c-1832-49f7-84fe-eadd2e6483d6",
  "user_type_id": "fcb3de16-32a1-4fc8-8b45-f9e6bb8964ca",
  "role_id": "913f5fb0-58ed-4a88-bcf0-leadc7e756d3",
  "status": "active",
  "created_at": "2025-12-16T05:55:43.734Z",
  "updated_at": "2025-12-16T05:55:43.734Z",
  "comp_code": "COMP1"
}
```



Download

2. To see all the accounts within a company, go to GET /accounts.

Note: managers can only see accounts within their department, while GET /accounts is not allowed for employees and clients

GET

/accounts

List all accounts

🔒 ⬆

Parameters

Cancel

Name	Description
comp_code	SUPER_ADMIN only — override tenant
string (query)	comp_code
limit	20
integer (query)	
offset	0
integer (query)	
search	search
string (query)	

Execute

Server response

Code

Details

200

Response body

```
{
  "id": "99f5600f-ef5d-441b-a8da-a6a65500b1c3",
  "emp_id": "EMP000",
  "fullname": "TestEmp",
  "username": "testemp",
  "email": "testemp@comp1.com",
  "department_id": "d89d399c-1832-49f7-84fe-eadd2e6483d6",
  "user_type_id": "fcb3de16-32a1-4fc8-8b45-f9e6bb8964ca",
  "role_id": "913f5fb0-58ed-4a88-bcf0-1eadc7e756d3",
  "status": "active",
  "created_at": "2025-12-16T05:55:43.734Z",
  "updated_at": "2025-12-16T05:55:43.734Z",
  "comp_code": "COMP1"
},
{
  "id": "53f502f0-ba23-4325-9a64-37e0f4f14fd5",
  "emp_id": "EMPLOYEE-012",
  "fullname": "Employee12Comp1",
  "username": "employee12comp1",
  "email": "employee12@comp1.com",
  "department_id": "d8ed4b50-a95e-4f28-9c2e-afd2aa6d6220",
  "user_type_id": "fcb3de16-32a1-4fc8-8b45-f9e6bb8964ca",
  "role_id": "913f5fb0-58ed-4a88-bcf0-1eadc7e756d3",
  "status": "active",
  "created_at": "2025-12-15T01:40:13.065Z",
  "updated_at": "2025-12-15T01:40:13.065Z"
}
```

Download

3. To see a specific account's details, copy the account id (in this case, the newly created TestEmp) and paste it into GET /accounts/{id}.

Note: managers can only use account ids within their department, while GET /accounts/{id} for employees and clients is only allowed for their own account ids

GET

/accounts/{id}

Get account by ID

🔒 ⬆

Parameters

Cancel

Name	Description
id required	
string(\$uuid) (path)	99f5600f-ef5d-441b-a8da-a6a65500b1c3

Execute

Server response

Code

Details

200

Undocumented Response body

```
{
  "id": "99f5600f-ef5d-441b-a8da-a6a65500b1c3",
  "emp_id": "EMP000",
  "fullname": "TestEmp",
  "username": "testemp",
  "email": "testemp@comp1.com",
  "department_id": "d89d399c-1832-49f7-84fe-eadd2e6483d6",
  "user_type_id": "fcb3de16-32a1-4fc8-8b45-f9e6bb8964ca",
  "role_id": "913f5fb0-58ed-4a88-bcf0-1eadc7e756d3",
  "status": "active",
  "created_at": "2025-12-16T05:55:43.734Z",
  "updated_at": "2025-12-16T05:55:43.734Z",
  "comp_code": "COMP1"
}
```

Download

4. To update any details about the account, copy the account id and paste it into PUT /accounts/{id}. In the request body, include any account details that need changing.

Note: comp_code is required, manager can only edit accounts within their department

PUT

/accounts/{id} Update account by UUID



Parameters

Cancel

Reset

Name	Description
id ★ required	
string(\$uuid)	99f5600f-ef5d-441b-a8da-a6a65500b1c3
(path)	

Request body

application/json

Edit Value

Schema

```
{
  "comp_code": "COMP1",
  "fullname": "TestEmp edit"
}
```

Server response

Code	Details
200	Response body <pre>{ "id": "99f5600f-ef5d-441b-a8da-a6a65500b1c3", "emp_id": "EMP000", "fullname": "TestEmp edit", "username": "testemp", "email": "testemp@comp1.com", "department_id": "d89c399c-1832-49f7-94fe-eadd2e6483d6", "user_type_id": "fcb3de16-32a1-4fc8-8b45-f9e6bb8964ca", "role_id": "913f5fb8-58ed-4a88-bcf8-leadc7e756d3", "status": "active", "created_at": "2025-12-16T05:55:43.734Z", "updated_at": "2025-12-16T06:48:07.295Z", "comp_code": "COMP1" }</pre>  Download

5. To disable an account, copy the account id and paste it into PATCH /departments/{id}/disable.

Note: manager can only disable accounts within their department

Name	Description
id ★ required	
string(\$uuid)	99f5600f-ef5d-441b-a8da-a6a65500b1c3
(path)	

Execute

Server response

Code	Details
200	Response body <pre>{ "id": "99f5600f-ef5d-441b-a8da-a6a65500b1c3", "status": "disabled", "updated_at": "2025-12-16T06:45:46.284Z", "comp_code": "COMP1" }</pre>  Download

6. To delete an account, copy the account id and paste it into DELETE /accounts/{id}.

The screenshot shows a REST client interface with a red header bar. The method is **DELETE** and the URL is **/accounts/{id}**. Below the URL bar, there's a "Parameters" section with a table. The table has two columns: "Name" and "Description". There is one parameter named "id" with a red asterisk and "required" text. The value is "99f5600f-ef5d-441b-a8da-a6a65500b1c3". Below the table is a blue "Execute" button. The "Server response" section shows a "Code" of 204 and "Details" for "Response headers". The headers are listed in a dark box.

Name	Description
id ★ required	string(\$uuid) (path)

Execute

Server response

Code: 204

Details: Response headers

```
access-control-allow-origin: *
connection: keep-alive
content-security-policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
cross-origin-opener-policy: same-origin
cross-origin-resource-policy: same-origin
date: Tue, 16 Dec 2025 06:49:03 GMT
keep-alive: timeout=5
origin-agent-cluster: ?1
referrer-policy: no-referrer
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-dns-prefetch-control: off
x-download-options: noopen
x-frame-options: SAMEORIGIN
x-permitted-cross-domain-policies: none
x-xss-protection: 0
```

Back-end File Descriptions:

- .env – defines environment variables for program
- package.json – contains basic program details and defines how program starts
- mobileLogin.js – basic mobile API for program
- README.txt – provides instructions for program setup
- sql
 - ddl.sql – provides SQL commands to reset data, create tables, user types, and roles with proper permissions
- src
 - server.js – creates and configures microservice API
 - swagger.js – creates interactive interface for microservice
 - config
 - index.js – turns .env variables into app settings
 - permissions.js – defines allowed actions of different account types
 - controllers
 - accountController.js – controls allowed account actions
 - authController.js – controls account login
 - departmentController.js – controls allowed department management actions
 - db
 - index.js – database access layer with multi-tenant isolation
 - middleware
 - auth.js – validates JWTs, enforces tenant isolation, and loads user info from database
 - errorHandler.js – catches and logs errors into JSON response
 - logger.js – logs HTTP requests and responses

- permission.js – enforces action-specific permissions, own-resource rules, and tenant/department isolation
 - validate.js – checks for validation errors and returns error 422 responses if any input is invalid
 - validator.js – collection of reusable express-validator helpers that trim, validate, and sanitize request fields
- routes (contains Swagger layout)
 - accounts.js – connects account-related HTTP requests to accountController.js
 - auth.js – connects to authController.js for user authentication and returns user info and JWT token
 - departments.js – connects department-related HTTP requests to departmentController.js
 - index.js – consolidates route modules into a single router
- services
 - accountService.js – handles account-related HTTP functions
 - authService.js – handles user authentication
 - departmentService.js – handles department-related HTTP functions
- tests
 - auth.rbac.test.js – automated test for authentication and RBAC logic